ASP.Net Core Middleware's

Overview



What is Middlewares?

-Http Request Response Flow

-Middleware Pipeline

-In-Built Middleware Order Flow

-Custom Middleware Example

-Different Purposes of Middleware

-Use(), Run() and Map()

What is Middlewares?

•In ASP.NET Core, Middleware is a piece of software that can handle an HTTP request or response.

• A given middleware component in ASP.NET Core has a very specific purpose. For example we may have a middleware component that authenticates a user, another piece of middleware to handle errors, yet another middleware to serve static files such as JavaScript files, CSS files, Images etc.

• It is this pipeline that determines how a request is processed. The request pipeline is configured as part of the application startup by the Program.cs file.

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

//To register services and dependencies

var app = builder.Build();

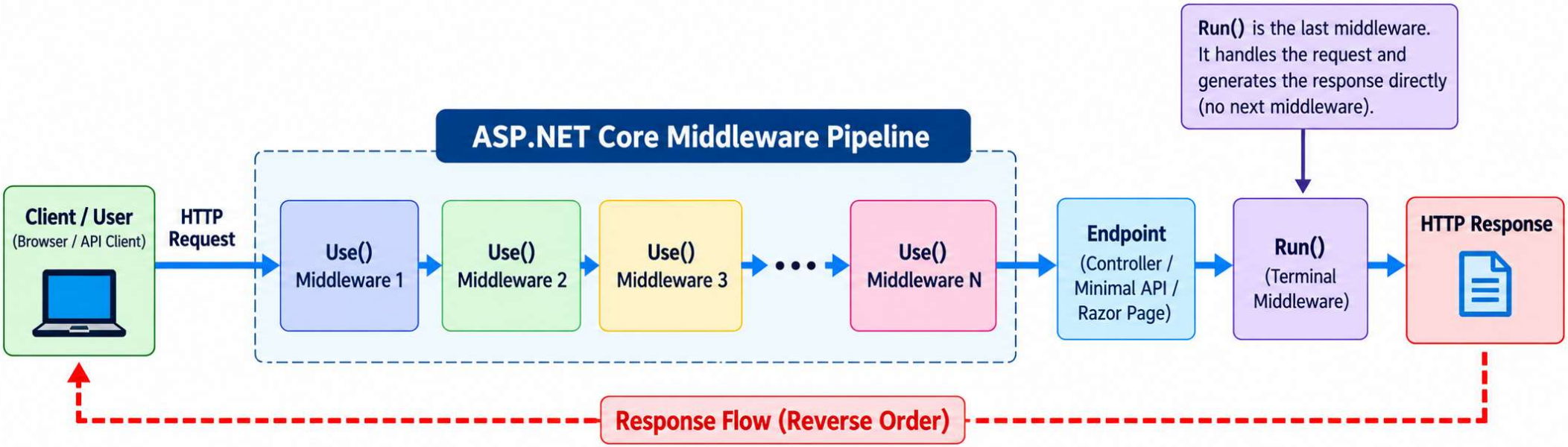
//in-built middleware goes here
if (builder.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}

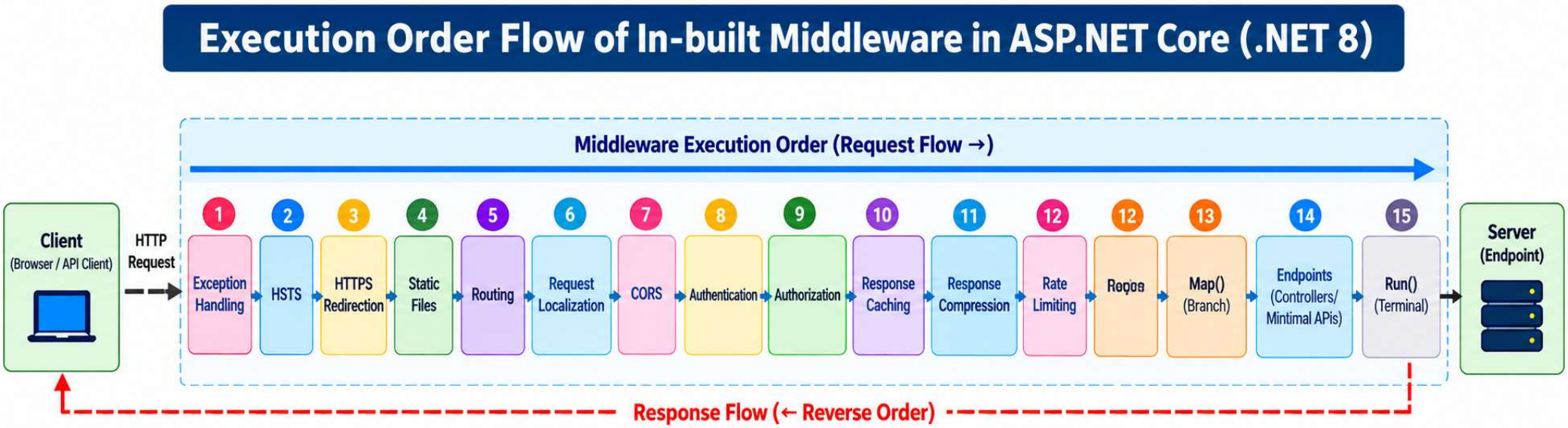
app.UseStaticFiles();
app.UseRouting();

//custom middleware goes here

app.Run();
```

- UseDeveloper ExceptionPage is one middleware and the second middleware is setup using the Run() method.


Run() is the last middleware. It handles the request and generates the response directly (no next middleware).
ASP.NET Core Middleware Pipeline
Client / User
(Browser / API Client)
HTTP Request
Use() Middleware 1
Use() Middleware 2
Use() Middleware 3
Use() Middleware N
Endpoint
(Controller / Minimal API / Razor Page)
Run()
(Terminal Middleware)
HTTP Response
Response Flow (Reverse Order)


Execution Order Flow of In-built Middleware in ASP.NET Core (.NET 8)
Middleware Execution Order (Request Flow →)
Client
(Browser / API Client)
HTTP
Request
1
Exception Handling
2
HSTS
3
HTTPS Redirection
4
Static Files
5
Routing
6
Request Localization
7
CORS
8
Authentication
9
Authorization
10
Response Caching
11
Response Compression
12
Rate Limiting
12
Regex
13
Map()
(Branch)
14
Endpoints
(Controllers/
Mintimal APIs)
15
Run()
(Terminal)
Server
(Endpoint)
Response Flow (← Reverse Order)

Custom Middleware Example

Suppose you want to create a middleware that logs information about each incoming HTTP request.

1. Create a new class for your custom middleware:

```
using System;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Http;

public class RequestLoggerMiddleware
{
    private readonly RequestDelegate _next;

    public RequestLoggerMiddleware(RequestDelegate
next)
    {
        _next = next;
    }
```

```
 public async Task InvokeAsync(HttpContext context)
   {
       // Log request information
       Console.WriteLine($"Request received:
{context.Request.Method} {context.Request.Path}");

       // Call the next middleware in the pipeline
       await _next(context);

       // You can add additional logic here that runs after the
response has been processed.
   }
}
```

2. Register the middleware in the Program.cs file.

```
using CustomMiddlewareDemo.CustomMiddleware;

var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

//in-built middleware goes here
if (builder.Environment.IsDevelopment())
{
   app.UseDeveloperExceptionPage();
}

app.UseStaticFiles();
app.UseRouting();

//To add custom middleware
app.UseMiddleware<RequestLoggerMiddleware>();

app.Run();
```

Essential In-Built Middleware's

Important In-built Middleware in ASP.NET Core (.NET 8)

ASP.NET Core provides many in-built middleware components to handle common cross-cutting concerns such as exception handling, routing, authentication, authorization, static files, etc.

Middleware	Purpose / What it does	Typical Usage / When to use	Example (Program.cs)
Exception Handling	Handles unhandled exceptions and returns a friendly error page or custom response.	• Use in production to handle errors gracefully. • Use different pages for Development vs Production.	`app.UseExceptionHandler("/Error");` `app.UseDeveloperExceptionPage(); // Dev only`
HSTS (HTTP Strict Transport Security)	Adds HSTS headers to enforce HTTPS connections.	• Use in production to improve security. • Avoid in development (not required).	`app.UseHsts();`
HTTPS Redirection	Redirects HTTP requests to HTTPS.	• Use when your application is running over HTTPS.	`app.UseHttpsRedirection();`
Static Files	Serves static files (CSS, JavaScript, images, etc.) from wwwroot (or configured folder).	• Use to serve static content like CSS, JS, images.	`app.UseStaticFiles();`
Routing	Matches incoming requests to endpoints (controllers, Razor pages, minimal APIs, etc.).	• Required for endpoint routing. • Works with UseAuthorization, UseEndpoints, MapControllers, etc.	`app.UseRouting();`
Authentication	Authenticates the user based on the configured scheme (e.g., JWT, Cookies, OpenID Connect).	• Use when authentication is enabled. • Must be added before UseAuthorization().	`app.UseAuthentication();`
Authorization	Checks if the authenticated user has required permissions.	• Use to enforce access control. • Must be added after UseAuthentication().	`app.UseAuthorization();`
Response Caching	Caches complete responses to improve performance.	• Use for cacheable, frequently accessed content.	`app.UseResponseCaching();`
Request Localization	Sets the request culture (language/region) based on configuration (query string, cookie, header, etc.).	• Use for multi-language/multi-region support.	`app.UseRequestLocalization();`
CORS	Enables Cross-Origin Resource Sharing to allow requests from different origins.	• Use when your API is accessed from different domains (e.g., SPA, frontend app).	`app.UseCors("MyPolicy");`
Session	Provides session state using a configured session store (in-memory, distributed, etc.).	• Use when you need to store user-specific data across requests.	`app.UseSession();`
Cookie Policy	Handles cookie consent and cookie security settings (SameSite, Secure, etc.).	• Use for GDPR/compliance and secure cookie handling.	`app.UseCookiePolicy();`
Response Compression	Compresses response body (e.g., Gzip, Brotli) to reduce response size.	• Use to improve performance for large responses.	`app.UseResponseCompression();`
Rate Limiting (New in .NET 7/8)	Limits the number of requests from a client to prevent abuse.	• Use to protect your APIs from overuse or DDoS.	`app.UseRateLimiter();`

Middleware for Path Mapping (Map)	Creates a separate middleware pipeline for a specific path (e.g., /api, /admin).	• Use when you need different middleware for different paths.	`app.Map("/api", apiApp => {`
			`apiApp.UseAuthentication();`
			`apiApp.MapControllers();`
			`});`

Terminal Middleware (Run)	Terminates the pipeline and generates the response directly (no next middleware).	• Used for simple responses or custom endpoints.	`app.Run(async context =>` `await context.Response.WriteAsync("Hello!");)`

Use(), Run() and Map()

Use() Method in ASP.NET Core Middleware – Workflow

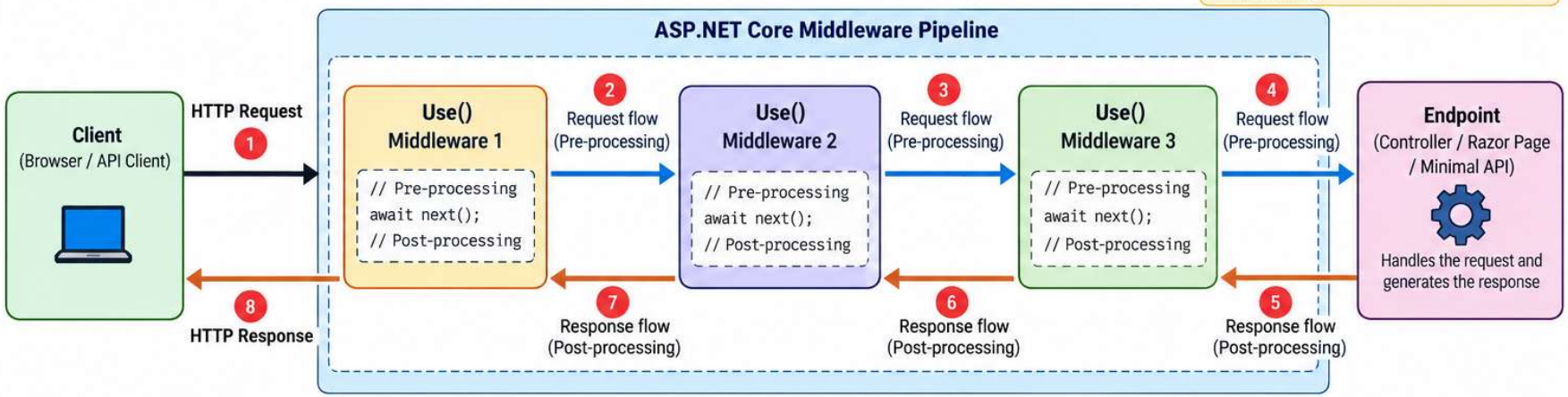
Registering middleware using Use() in Program.cs

```
var app = builder.Build();

app.Use(async (context, next) => { /* Middleware 1 */ });
app.Use(async (context, next) => { /* Middleware 2 */ });
app.Use(async (context, next) => { /* Middleware 3 */ });
app.MapControllers();
app.Run();
```

What does Use() do?

- Adds a middleware component to the request pipeline.
- Each middleware can perform some work before calling the next middleware.
- Uses the next delegate to pass the request to the next middleware.
- After the next middleware (and endpoint) completes, control returns to the previous middleware to perform post-processing (response).



1 Request Arrives
Client sends an HTTP request to the ASP.NET Core application.

2 Middleware 1 (Pre-processing)
Performs some work (e.g., logging, auth) and calls next() to pass the request to the next middleware.

3 Middleware 2 (Pre-processing)
Performs its work and calls next().

4 Middleware 3 (Pre-processing)
Performs its work and calls next() to pass the request to the endpoint (controller / minimal API).

5 Endpoint Execution
The endpoint handles the request and generates a response.

6 Middleware 3 (Post-processing)
After the endpoint completes, control returns to Middleware 3 to perform post-processing (e.g., modify response, logging).

7 Middleware 2 (Post-processing)
Control returns to Middleware 2 for post-processing.

8 Middleware 1 (Post-processing)
Control returns to Middleware 1 for post-processing and finally the response is sent back to the client.

Map() Method in ASP.NET Core Middleware – Workflow

Configuring Map() in Program.cs

```
var app = builder.Build();

app.Use(async (context, next) =>
{   // Global middleware - runs for all requests
    await next();
});

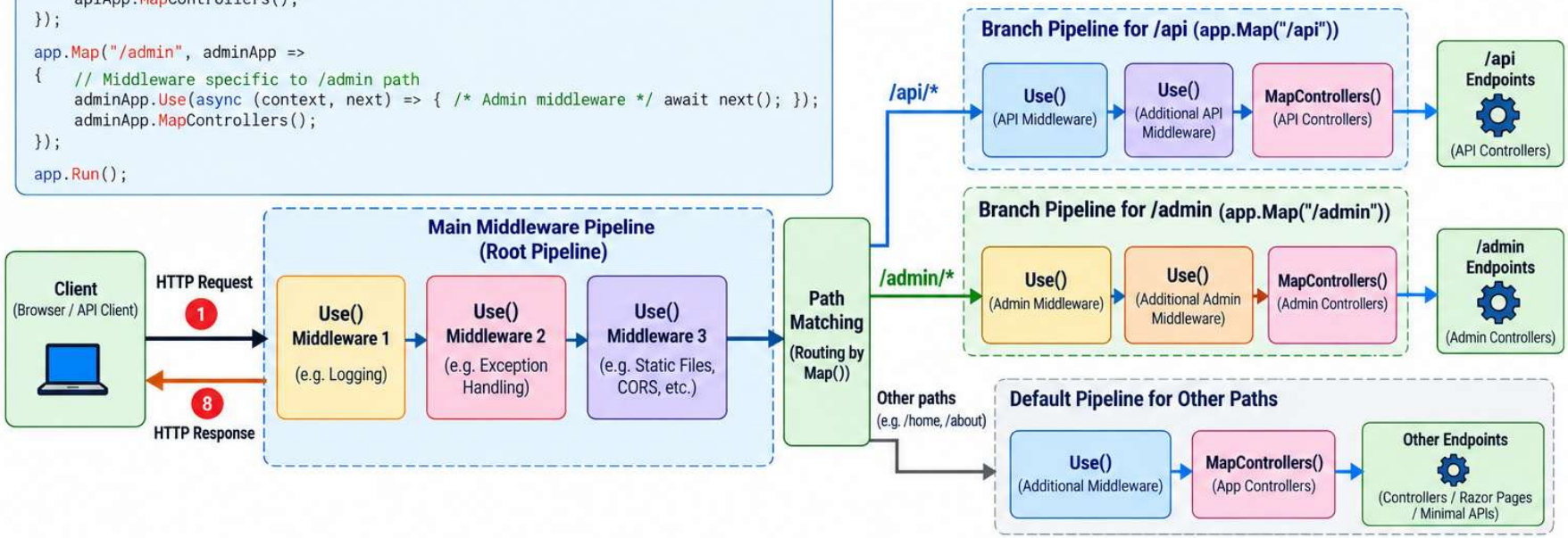
app.Map("/api", apiApp =>
{   // Middleware specific to /api path
    apiApp.Use(async (context, next) => { /* API middleware */ await next(); });
    apiApp.MapControllers();
});

app.Map("/admin", adminApp =>
{   // Middleware specific to /admin path
    adminApp.Use(async (context, next) => { /* Admin middleware */ await next(); });
    adminApp.MapControllers();
});

app.Run();
```

What does Map() do?

- Creates a branch in the middleware pipeline for a specific request path (path prefix).
- Requests matching the specified path are handled by a separate, independent pipeline.
- Only the middleware registered inside that Map() branch will run for matching requests (along with global middleware).
- Requests that do not match the path will skip that branch and continue with the main pipeline.



1 Request Arrives
Client sends an HTTP request to the ASP.NET Core application (e.g. /api/products).

2 Global Middleware Runs
Middleware registered using Use() in the main pipeline executes for all requests (e.g. logging, exception handling, etc.).

3 Path Matching (Map())
The framework checks if the request path matches any Map() path (e.g. /api, /admin).

4 Branch Pipeline Executes
If the path matches, the request is processed by the corresponding branch pipeline (only the middleware inside that Map() runs).

5 Endpoint Execution
The endpoint (controller / minimal API / Razor page) for that branch handles the request and generates a response.

6 Response Returns
The response flows back through the same branch middleware (post-processing) and then through the global middleware to the client.

Run() Method in ASP.NET Core Middleware – Workflow

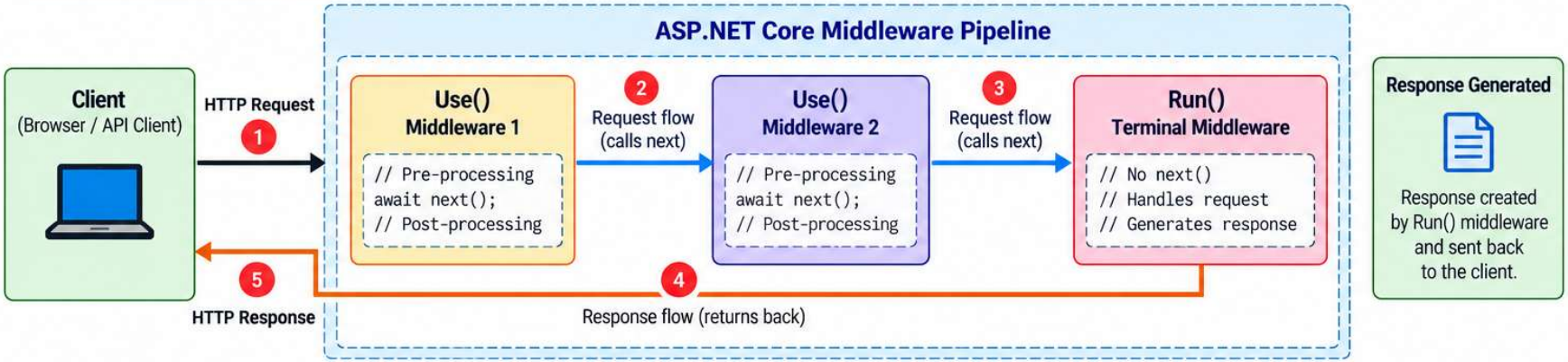
Configuring Run() in Program.cs

```
var app = builder.Build();
app.Use(async (context, next) =>  // Middleware 1
{
    // Pre-processing
    await next();
    // Post-processing
});

app.Run(async context =>
{   // Terminal middleware - generates the response
    await context.Response.WriteAsync("Hello from Run() terminal middleware!");
});
```

What does Run() do?

- Adds a terminal middleware to the pipeline.
- Does NOT take a next delegate.
- Handles the request and generates the response directly.
- Once Run() is executed, no subsequent middleware is called because it does not call next().
- Typically used at the end of the pipeline for simple scenarios (e.g., returning a fixed response).



1 Request Arrives

Client sends an HTTP request to the ASP.NET Core application.

2 Middleware 1 (Use)

Performs pre-processing (e.g., logging).
Calls next() to pass the request to the next middleware.

3 Middleware 2 (Use)

Performs its work.
Calls next() to pass the request to the next middleware (Run()).

4 Run() Middleware (Terminal)

Handles the request directly and generates the response.
Does NOT call next().
No further middleware is executed after this.

5 Response Returns

The response generated by Run() flows back through the previous middleware (post-processing) and is sent to the client.

Method	Purpose	Can continue pipeline?
Use()	Performs some work and can call the next middleware	✅ Yes
Run()	Terminal middleware; does not call next middleware	❌ No
Map()	Creates a separate pipeline for a specific URL/path	Depends on branch

References

https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/

https://www.tutorialsteacher.com/core/aspnet-core-middleware

https://www.dotnetcurry.com/aspnet-core/middleware-in-aspnetcore

https://www.c-sharpcorner.com/article/overview-of-middleware-in-asp-net-core/

https://www.tutorialspoint.com/asp.net_core/asp.net_core_middleware.htm

https://endjin.com/blog/2022/02/understanding-middleware-in-aspnet-core

https://dotnettutorials.net/lesson/asp-net-core-middleware-components/